Problem A. Journey of SYSU

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

This year is the 100th anniversary of the founding of Sun Yat-sen University. In order to show the commemoration of this anniversary, the ACMM Association decided to add elements related to the anniversary in this contest.

You are given a string that is exactly a multiple of four in length and consists of uppercase letters only. You are allowed a number (possibly zero) of operations, each of which allows you to select a letter of the string and modify it to any uppercase letter.

Your final goal is to modify this string to a string with `SYSU` as a period (e.g. `SYSU`, `SYSUSYSU`, `SYSUSYSUSYSU`). What is the minimum number of operations required to accomplish this goal?

Input

The first line contains a positive integer n ($4 \le n \le 10^5$, n is a multiple of 4), representing the length of the string.

The second line contains a string with length n and consists of uppercase letters only, representing the string given to you.

Output

Output a single integer, denoting the answer.

Examples

standard input	standard output
12 SPSKBYSHQASY	7
12 SYSUSYSUSYSU	0

Note

In the first example, you need to change at least 7 positions to make the string equal to `SYSUSYSUSYSU`.

In the second example, the string itself satisfies the requirement, so no operation is needed.

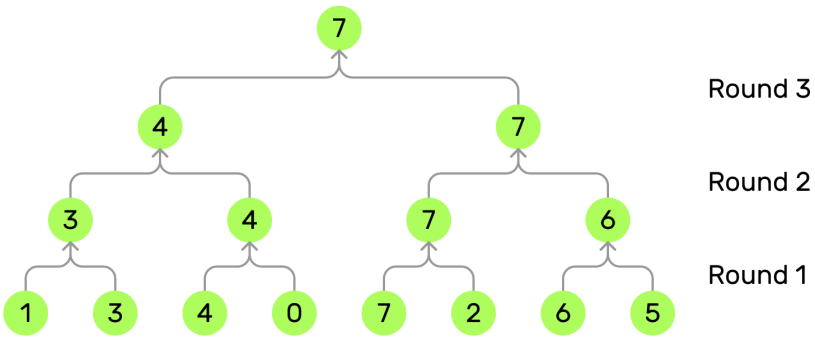
Problem B. WAduck and Cuteness Contest

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

WAduck is a cute duck. One day, he registers for a cuteness contest.

There are 2^n ducks in the contest. Each of them has a cuteness value, which is an integer between 0 and $2^n - 1$. Additionally, the cuteness values are pairwise different. WAduck's cuteness value is k.

The contest is played in a knockout format. The contest will run for n rounds. In each round, all ducks are paired based on the bracket. For each pair, the duck with larger cuteness wins and moves to the next round, and the other loses and is eliminated from the contest.



An example of a cuteness contest.

It's obvious that the duck with cuteness $2^n - 1$ will be the winner. However, the bracket for the contest is unknown before the contest, so the ducks don't know their opponents in each round. WAduck wonders how many times he can win at most.

Input

The only line contains two integers n and k. It's guaranteed that $1 \le n \le 30$, $0 \le k \le 2^n - 1$.

Output

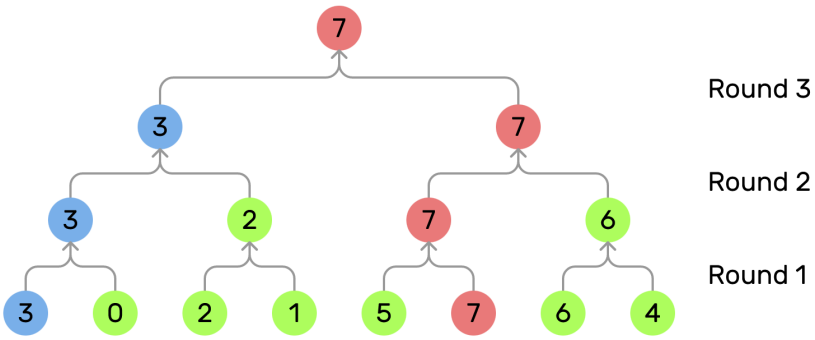
Output a single integer, denoting your answer.

Examples

standard input	standard output
3 3	2
3 7	3

Note

A possible situation when $n = 3$, which matches the examples:


7
3
7
3
2
7
6
3
0
2
1
5
7
6
4
Round 3
Round 2
Round 1

Problem C. WAduck Rotates For One Minute

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

WAduck is a funky duck, and he has n coins. Each side of the coins has a number, which is an integer between 1 and n. Numbers on two sides can be the same or different. Also, each integer between 1 and n appears in these n coins **exactly twice** in total.

WAduck wants to play a game with these coins. At first, the n coins are placed face up on the table. Then WAduck chooses some of the coins (possibly none of them) and flips them. After that, WAduck can see n numbers on the table. If each integer between 1 and n appears **exactly once**, WAduck will be very thrilled and rotate for one minute.

WAduck wonders how many kinds of choices to flip coins can make him thrilled. As the answer can be very large, you need to output the answer modulo 1 000 000 007.

Input

The first line contains one integer n ($1 \le n \le 10^5$), denoting the number of coins.

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le n$), denoting the number of the i-th coin on heads.

The third line contains n integers $b_1, b_2, \ldots, b_n$ ($1 \le b_i \le n$), denoting the number of the i-th coin on tails.

It's guaranteed that each integer between 1 and n appears **exactly twice** in $\{a_i\}$ and $\{b_i\}$.

Output

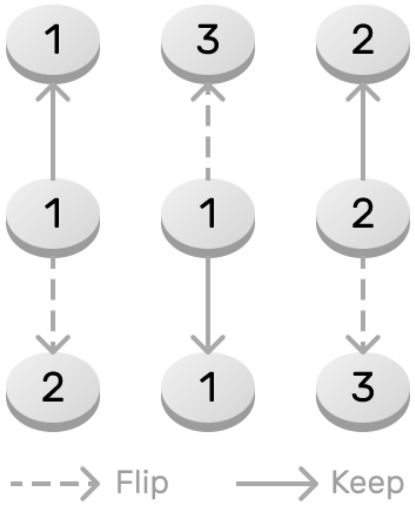
Output a single integer, denoting your answer modulo 1 000 000 007.

Examples

standard input	standard output
3 1 1 2 2 3 3	2
2 1 2 1 2	4

Note

In the first example, there are 2 choices:



In the second example, whatever WAduck flips, the result can always make WAduck thrilled, so there are 4 choices.

Problem D. WAduck Gets Wrong Answer

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

WAduck's English is bad.

One day, WAduck takes an English test. There are K questions and N options. Every question has a correct answer among these N options, and **no two correct answers are the same**. WAduck should write an option to each question, and **no two options written are the same either**. WAduck can get 1 point from each question if his answer to the question matches the correct answer.

Here is an example. Assume that the options are `A,B,C,D,E,F,G`. If the correct answer is `BCGFA` and WAduck's answer is `BAGFD`, then WAduck gets 3 points.

Unfortunately, WAduck gets 0 points in the end, so he wants to know how many different answers will get 0 points when the correct answer is fixed. As the number can be very large, you need to output the answer modulo 1 000 000 007.

Two answers are considered different if they give different options to at least one question.

Input

One line contains two integers N and K ($1 \le K \le N \le 10^5$).

Output

Output a single integer, denoting the number of answers modulo 1 000 000 007.

Example

standard input	standard output
4 2	7

Note

In the example, we can assume the correct answer is `AB`, then there are 7 answers that get 0 points: `BA,BC,BD,CA,CD,DA,DC`.

Problem E. Shield Block

Input file:	`standard input`
Output file:	`standard output`
Time limit:	2 seconds
Memory limit:	256 megabytes

Little Q is playing **Hearthstone**, and his opponent is a Warrior who is skilled at getting armor.



Initially, there are n cards in Little Q's deck, and he will play them **from top to bottom**. The i-th card deals a_i damage to the Warrior.

Each turn, the Warrior will **reset** his armor to D points (note that it is "reset", not "increase"). Then, Little Q will play k cards in order (If the remaining cards are fewer than k, Little Q will still play all the remaining cards in the last turn). If the total damage of these k cards (called S) exceeds D, the Warrior will take $S - D$ damage; otherwise, the Warrior will take no damage. The game ends when all the cards in Little Q's deck have been played.

Little Q has a special ability: he can remove any cards from his deck before the game starts, without changing the relative position of the remaining cards. He wants to know, for each $k = 1, 2, ..., n$, the maximum damage he can deal to the Warrior.

Input

The first line contains two integers n, D ($1 \le n \le 10^6, 1 \le D \le 10^9$), representing the number of cards in Little Q's deck and the Warrior's armor.

The next line contains n integers a_i ($1 \le a_i \le 10^{12}$), representing the damage of each card in the deck **from top to bottom**.

Output

Output n lines. The i-th line contains an integer, denoting the maximum damage Little Q can deal to the Warrior when $k = i$.

Example

standard input	standard output
5 5 2 1 5 4 3	0 4 7 9 10

Note

In the example, when $k = 1$, since only one card can be played per turn, it is impossible to deal any damage to the Warrior anyway.

When $k = 2$, by removing the 2nd card, the damage in the first round is $2 + 5 - 5 = 2$, and in the second round, the damage is $4 + 3 - 5 = 2$.

When $k = 3$, by removing the 1st and 2nd cards, the damage in the first round is $5 + 4 + 3 - 5 = 7$.

When $k = 4$, by removing the 2nd card, the damage in the first round is $2 + 5 + 4 + 3 - 5 = 9$.

When $k = 5$, no cards are removed, and the damage in the first round is $2 + 1 + 5 + 4 + 3 - 5 = 10$.

Problem F. Pac-Girl

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

Little Q is playing Celeste, controlling Madeline to escape from three Badelines. Madeline is initially placed at the origin, and for every moment, Little Q can move Madeline by one unit in one of the four directions: up, down, left, or right (we do not consider gravity, meaning Madeline will not naturally fall down).

Three Badelines will be generated at the origin at the first 3 moments and follow the same trace as Madeline (in other words, Badelines will appear in the same position as Madeline's last **three** positions). For every moment, Badelines will move or generate after Madeline's move. Madeline cannot move to a position where Badelines are located.



The picture shows how Badelines generate and move during the first 4 moments.

Little Q wants to know how many different operation plans there are such that Madeline will not collide with Badelines in n moments. Two operation plans are considered different if, at any moment, Little Q moves Madeline in a different direction.

Since the number of plans may be very large, Little Q only wants to know the answer modulo 1 000 000 007 (i.e., the remainder when the answer is divided by 1 000 000 007).

Input

The only line contains an integer n ($1 \le n \le 10^6$).

Output

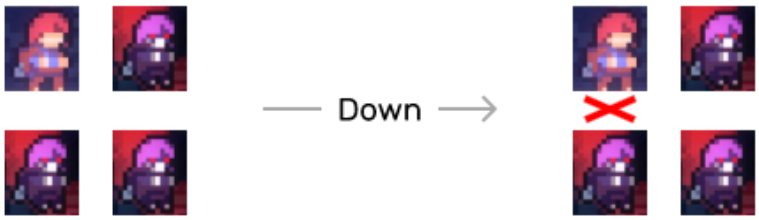
Output a single integer representing the answer modulo 1 000 000 007.

Examples

standard input	standard output
2	12
3	36
4	100

Note

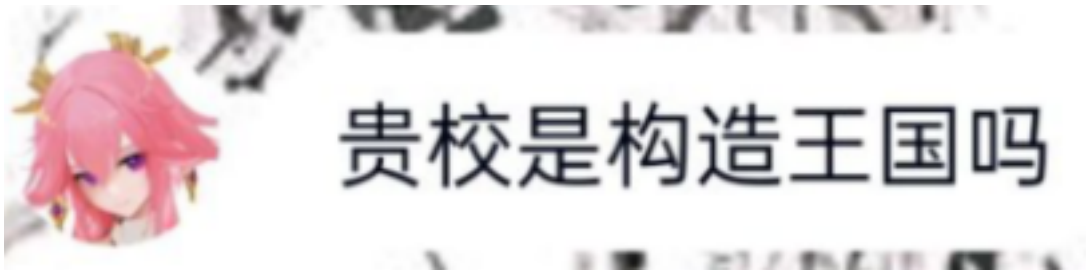
When $n = 4$, an invalid operation plan is (Right, Up, Left, Down) because, as shown in the picture below, Madeline will collide with a Badeline that is in the position she occupied three moments ago.



Problem G. Make SYSU Great Again 3.0 Alpha

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

As you all know, you are participating in the SYSUCPC (SYSU Constructive Problem Contest) qualifiers. Undoubtedly, the Constructive Kingdom of Sun Yat-sen University is eager for you to tackle some constructive problems.



Little Q is stargazing. He discovers that *Regulus* consists of n stars and m star paths connecting pairs of stars. Since *Regulus* is a galaxy, the m star paths will connect all the stars together (ensure that every pair of stars can reach each other through bidirectional star paths) . However, the star paths can only transmit light **in one direction**. Little Q does not know the direction of light transmission for each path, and he only knows the **brightness** of each star.

The **brightness** of the i-th star is a_i, meaning that starting from the i-th star, there exists at least one light path of length a_i. Now, Little Q wants you to tell him a possible direction in which each star path transmits light.

Input

The first line contains two integers n and m ($1 \le n \le 10^5, 1 \le m \le 2n$), representing the number of stars and star paths, respectively.

The second line contains n integers a_i ($0 \le a_i \le 10^9$), representing the brightness of each star.

The next m lines each contain two positive integers x_i and y_i ($1 \le x_i, y_i \le n$), indicating that the i-th star path connects the x_i-th and y_i-th stars.

It is guaranteed that no star path connects the same star to itself, and no two star paths connect the same pair of stars.

Output

Output a string of length m consisting only of `0` and `1`. If the i-th character is `0`, it means that the i-th star path transmits light in the direction $x_i \to y_i$; otherwise, it transmits light in the direction $y_i \to x_i$.

If there are more than one solutions, output any of them. If there is no solution, output "-1" (without quotes).

Examples

standard input	standard output
6 7 1 2 3 4 5 6 1 2 2 3 3 4 4 5 5 6 6 4 5 3	0000001
5 4 1 0 0 2 0 1 2 2 3 3 4 2 5	0110
5 4 0 2 0 2 3 4 2 5 4 5 1 3 4	-1

Note

The picture below shows the *Regulus* given by the second example.

